

UNIVERSIDAD DE BUENOS AIRES

Facultad de Ingeniería

Redes (TA048)

Trabajo Práctico N°1: File Transfer

Transferencia Confiable de Archivos sobre UDP (Stop & Wait y TCP con SACK)

Integrantes

Nombre y Apellido	Padrón	Correo Electrónico
Ramiro Barcala Roca	112478	rbarcala@fi.uba.ar
Francisco Monopoli	104667	fmonopoli@fi.uba.ar
Valentina Sol Arroyo	112294	vsarroyo@fi.uba.ar
Atuel Sebastian Fullana	110247	afullana@fi.uba.ar
Elias Edgar Mendoza	104607	emendoza@fi.uba.ar

Docentes: Esteban Carisimo y Juan Ignacio Lopez Pecora

Fecha de Entrega: 6 de octubre de 2026

Segundo Cuatrimestre de 2026

Índice

1. Introducción	2
1.1. Objetivos del Proyecto	2
1.2. El Problema de la Transferencia Confiable sobre UDP	2
2. Hipótesis y Suposiciones Realizadas	3
3. Implementación	4
3.1. Estructura y Arquitectura del Sistema	4
3.2. Protocolo de Capa de Aplicación	4
3.3. Mecanismos de Transferencia de Datos Confiable	4
3.3.1. Variante 1: Stop & Wait	4
3.3.2. Variante 2: TCP con SACK (Selective Repeat / Recovery)	4
3.4. Manejo de Concurrencia en el Servidor	5
3.5. Manejo de Flujo de Errores y Casos Contemplados	5
4. Pruebas	6
4.1. Topología y Entorno en Mininet	6
4.2. Upload sin Pérdida de Paquetes y RTT de 40 ms	7
4.2.1. Variante Stop & Wait	7
4.2.2. Variante SACK	8
4.3. Concurrencia	8
4.4. Inspección de Tráfico y Eficiencia en Wireshark	10
4.5. Validación de Requisitos Críticos de Rendimiento	10
4.5.1. Cumplimiento del requisito crítico: Transferencia de 5 MB en menos de 2 minutos	10
5. Preguntas a Responder	11
5.1. Pregunta 1: Arquitectura Cliente-Servidor	11
5.2. Pregunta 2: Función de un Protocolo de Capa de Aplicación	11
5.3. Pregunta 3: Detalle del Protocolo de Aplicación Desarrollado	11
5.4. Pregunta 4: Características y Comparativa de TCP y UDP	12
5.5. Pregunta 5: Mecanismos de Control de Congestión	12
5.6. Pregunta 6: Comparación de Tiempos entre Protocolos	12
6. Dificultades Encontradas	14
7. Conclusión	15

1. Introducción

1.1. Objetivos del Proyecto

El objetivo del trabajo práctico es implementar un protocolo que permita la descarga y la carga de archivos y provea el servicio de Transferencia de Datos Confiable (RDT). Para cumplir con el objetivo implementamos dos formas de transmitir los paquetes: *Stop & Wait* y *Error Recovery* (TCP con SACK). Utilizamos la arquitectura Cliente-Servidor, Mininet para simular una red real con retrasos (delay) y pérdidas de paquetes, y Wireshark para capturar el tráfico y comparar el desempeño de ambos mecanismos de transferencia, sumandole una extension para ver los campos de los paquetes que le llegan al server en Mininet.

1.2. El Problema de la Transferencia Confiable sobre UDP

El protocolo UDP (User Datagram Protocol) ofrece un modelo de servicio en la capa de transporte que es no orientado a la conexión y del tipo *best-effort*. Esto implica que trata a cada datagrama de manera completamente independiente, sin garantizar su entrega, sin asegurar el orden de llegada y permitiendo la posible pérdida o duplicación de los paquetes a través de la red. Si bien esto reduce drásticamente la latencia y la sobrecarga en los headers, resulta insuficiente por sí solo para aplicaciones que requieren la transferencia íntegra, exacta y libre de corrupción de un archivo.

Para sortear las limitaciones inherentes de UDP y lograr una Transferencia de Datos Confiable (RDT), resulta indispensable implementar una serie de mecanismos de control directamente en la capa de aplicación. Las problemáticas principales que se debieron resolver incluyen:

- **Detección de pérdidas mediante temporizadores (*Timeouts*):** Al no existir garantías de entrega por parte de la red, el emisor debe ser capaz de deducir que un paquete de datos (o su respectiva confirmación) se ha perdido. Esto se soluciona inicializando un temporizador al momento de enviar un segmento y disparando una retransmisión automática si el tiempo límite expira sin haber recibido una respuesta.
- **Reordenamiento y control de duplicados mediante números de secuencia:** Dado que los routers intermedios pueden alterar el orden de los datagramas o duplicarlos, la capa de aplicación debe inyectar un sequence number único en el header de cada paquete. Esto le proporciona al receptor la capacidad de reensamblar los fragmentos del archivo en el orden original y de descartar aquellos segmentos que ya hayan sido procesados.
- **Confirmación de recepción (ACKs y SACK):** Es importante establecer un canal de feedback donde el receptor notifique al emisor sobre los datos que llegaron con éxito. En nuestra implementación, esto se aborda mediante ACK's, que indican el próximo sequence number esperado, y mediante SACK's, que proveen información detallada sobre los bloques recibidos fuera de orden para habilitar una recuperación de errores mucho eficiente.

2. Hipótesis y Suposiciones Realizadas

Para el desarrollo de los protocolos se tomaron en cuenta las siguientes hipótesis y suposiciones:

- **Compatibilidad de protocolos:** El servidor y los clientes usan el mismo protocolo.
- **Sobreescritura de archivos en el servidor:** Al recibir un archivo con un nombre que ya existe en el servidor, se reescribe el contenido del archivo previo.
- **Sobreescritura de archivos en el cliente:** Al descargar y escribir un archivo con un nombre preexistente en el cliente, se reescribe su contenido.
- **MTU y prevención de fragmentación:** El MTU es de 1500 bytes.
- **Temporización fija (Timeout):** El *Timeout* de retransmisión se define de forma estática, pero garantizando un margen suficiente para que el protocolo funcione correctamente en enlaces con RTT de hasta 300 ms.
- **Integridad de datos en almacenamiento local:** Se asume soporte para lectura y escritura binaria (`'rb'`, `'wb'`) en ambos extremos para asegurar transferencias exactas bit a bit.

3. Implementación

Para poder lograr una Transferencia de Datos Confiable sobre UDP, fue necesario implementar a nivel de aplicación funciones características de TCP, tales como el reenvío y retransmisión de paquetes ante pérdidas, *handshake* para la sincronización inicial y el cierre ordenado de la conexión.

3.1. Estructura y Arquitectura del Sistema

El proyecto está organizado en una estructura modular para separar la interfaz de usuario por línea de comandos (CLI) de la lógica de red subyacente. En el directorio raíz se proveen los ejecutables : `src/upload` (inicializa el cliente de subida), `src/download` (cliente de bajada) y `src/start-server` (servidor de almacenamiento). Toda la lógica central y compartida reside en el directorio `src/lib/`. Dentro de este módulo se abstraio la creación de sockets, el empaquetado y desempaquetado de mensajes (*parsing*), el manejo de sesiones (`session.py`) y la implementación particular de cada mecanismo de transferencia de datos confiable (`stop_and_wait.py` y `sack.py`). Finalmente, se incluye un archivo `README.md` documentando las instrucciones de ejecución.

3.2. Protocolo de Capa de Aplicación

Campo	Tamaño (bytes)	Descripción / Función
Type	1	Tipo de mensaje (ej: 0=DATA, 1=ACK, 2=SACK, 3=ERROR, 4=FIN)
SeqNum	4	Número de secuencia del segmento
AckNum	4	Número de confirmación acumulativo
SackData	Variable	Información de bloques fuera de orden recibidos (si aplica)
Length	2	Longitud útil de la carga útil (payload)
Payload	Variable	Bloque binario de datos del archivo transferido

Tabla 1: Definición del header del protocolo implementado.

3.3. Mecanismos de Transferencia de Datos Confiable

3.3.1. Variante 1: Stop & Wait

El mecanismo de *Stop & Wait* implementado opera bajo un ciclo estricto y sincrónico. El emisor despacha un único datagrama hacia la red y transiciona inmediatamente a un estado de bloqueo, donde inicializa un temporizador (*timeout*) a la espera del ACK correspondiente enviado por el receptor. Si el ACK con el sequence number esperado llega a tiempo, el emisor envia el datagrama siguiente. En caso de pérdida del paquete o del ACK, el temporizador termina y el protocolo retransmite el último datagrama enviado. Este mecanismo resulta simple y asegura la entrega ordenada, pero aumenta el Throughput ya que permite un solo paquete simultáneo.

3.3.2. Variante 2: TCP con SACK (Selective Repeat / Recovery)

Para optimizar el uso del canal de comunicación, se implementó el mecanismo con soporte SACK (*Selective Acknowledgment*). A diferencia del modelo anterior, aquí se utiliza una ventana deslizante de un tamaño fijo predeterminado (32 paquetes). El emisor puede enviar múltiples datagramas de manera consecutiva hasta agotar el espacio de su ventana. El receptor avisa que le llegó devolviendo un ACK acumulativo (indicando el último paquete recibido en orden) y añade los bloques SACK que referencian los paquetes recibidos con secuencias posteriores fuera de orden (generando huecos). Al recibir estos datos, el emisor implementa una retransmisión selectiva, reenviando únicamente aquellos paquetes que el receptor indicó no recibir, logrando una transferencia segura sin reenviar paquetes de mas.

3.4. Manejo de Concurrency en el Servidor

Dado que el servidor debe permanecer a la escucha en un único puerto UDP, las peticiones entrantes llegan todas a un mismo socket. Para procesarlas concurrentemente, el servidor lee el header de los datagramas entrantes y extrae la tupla de (IP, Puerto) del emisor. Si se trata de un nuevo cliente enviando un (START), se le asigna una sesión para manejar dicha transferencia (mediante un hilo independiente). Si es un paquete de un flujo ya existente, se rutea dicho segmento directamente a la sesión activa correspondiente. Esto garantiza que las variables de estado, las ventanas y los *timeouts* de múltiples clientes operen simultáneamente de forma segura y sin mezclar datos.

3.5. Manejo de Flujo de Errores y Casos Contemplados

1. **Condición de error 1:** Archivo inexistente en el servidor ante una solicitud de DOWNLOAD, mostrando en un log si no se pudo recibir el archivo (inexistente).
2. **Condición de error 2:** *Timeout* crítico por pérdida sostenida de paquetes o caída abrupta del host remoto, también avisando mediante logs cuando se cierra la conexión por estas causas.

4. Pruebas

4.1. Topología y Entorno en Mininet

Para llevar a cabo las pruebas se utilizó la siguiente topología en Mininet, la cual simula 5 hosts: 4 de ellos actúan como clientes y 1 host funciona como el servidor central. Todos los hosts se encuentran interconectados a través del mismo switch. El *delay* y el *packet loss* configurados en los enlaces varían de acuerdo a los requerimientos de cada prueba.

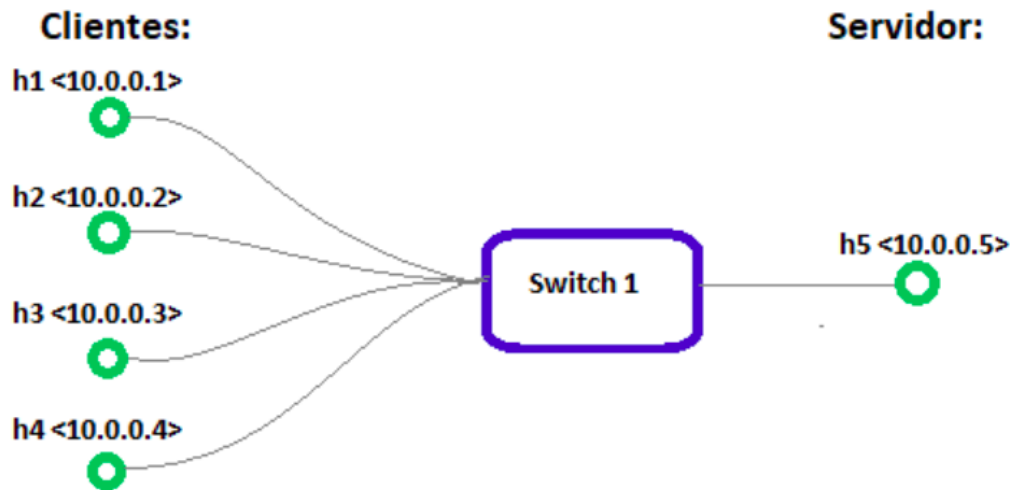


Figura 1: Topología utilizada en Mininet compuesta por 4 clientes y 1 servidor conectados a un switch.

Para levantar esta topología se ejecuta el siguiente comando:

```
atuelfullana@compu-atu:~/Desktop/redes/TP1 (main)$ make run-mininet
sudo mn --topo single,2 --link tc,loss=10,delay=20ms
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(20ms delay 10.00000% loss) (20ms delay 10.00000% loss) (h1, s1) (20ms delay 10.00000% loss) (20ms delay 10.00000% loss) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
*** Starting 1 switches
s1 ... (20ms delay 10.00000% loss) (20ms delay 10.00000% loss)
*** Starting CLI:
mininet> 
```

Figura 2: Comando de inicio del script de la topología en Mininet.

A modo de referencia, la sintaxis y comandos utilizados para ejecutar tanto el servidor como las aplicaciones cliente de subida y descarga son los siguientes:

```
# Comando de ejemplo para iniciar el servidor
python start-server -H 10.0.0.1 -p 8080 -s ./storage -v
```

```
# Comando de ejemplo para ejecutar upload cliente
python upload -H 10.0.0.1 -p 8080 -s ./archivos/test.bin -n test.bin -r
sack -v

# Comando de ejemplo para ejecutar download cliente
python download -H 10.0.0.1 -p 8080 -d ./descargas/test.bin -n test.bin -r
stop-wait -v
```

Dentro de la carpeta `/tmp/` se tienen guardados varios archivos de prueba de distintos tamaños y extensiones (`.bin` y `.txt`) para ser transmitidos durante las mediciones:

```
(venv) rbarcala@rbarcala-B250M-D3H:~/MEGA/Redes/TP1$ # Crear directorio temporal
mkdir -p /tmp/archivos_prueba
mkdir -p /tmp/storage

# Archivo de 500 KB
dd if=/dev/urandom of=/tmp/archivos_prueba/500KB.bin bs=1K count=500

# Archivo de 2 MB
dd if=/dev/urandom of=/tmp/archivos_prueba/2MB.bin bs=1M count=2

# Archivo de 5 MB
dd if=/dev/urandom of=/tmp/archivos_prueba/5MB.bin bs=1M count=5
500+0 records in
500+0 records out
512000 bytes (512 kB, 500 KiB) copied, 0.00429631 s, 128 MB/s
2+0 records in
2+0 records out
2097152 bytes (2.1 MB, 2.0 MiB) copied, 0.010704 s, 210 MB/s
5+0 records in
5+0 records out
5242880 bytes (5.2 MB, 5.0 MiB) copied, 0.0180344 s, 291 MB/s
```

Figura 3: Archivos de prueba disponibles en el directorio `/tmp/`.

4.2. Upload sin Pérdida de Paquetes y RTT de 40 ms

4.2.1. Variante Stop & Wait

```
"Node: h5"
root@compu-atu:/home/atuefullana/Desktop/redes/TP1/src# python3 start-server -
H 10.0.0.5 -p 5005
FIN recibido: seq=489
Transferencia completada: 500000 bytes
```

Figura 4: Puesta en marcha del servidor en el host `h5`.

Primero se enciende el servidor (en este caso alojado en el host `h5`). Luego, desde uno de los clientes se ejecuta el comando para solicitarle al servidor la subida de un archivo utilizando el protocolo *Stop & Wait*.

```
"Node: h2"
root@compu-atu:/home/atuefullana/Desktop/redes/TP1/src# python3 upload -H 10.0.0.5 -p 5005 -
s /tmp/archivo-500KB.bin -n archivo-500KB.bin -r sw
File '/tmp/archivo-500KB.bin' sent successfully using Stop-and-Wait protocol.
root@compu-atu:/home/atuefullana/Desktop/redes/TP1/src#
```

Figura 5: Ejecución de la subida de archivo desde el cliente mediante *Stop & Wait*.

Una vez finalizada la subida del archivo, el cliente recibe un mensaje (mediante logs) notificando que el archivo se subió exitosamente utilizando el protocolo *Stop & Wait*. Por su parte, el servidor imprime en consola un mensaje que resume la cantidad de bytes y paquetes transferidos.


```
"Node: h2"
root@compu-atu:/home/atuefullana/Desktop/redes/TP1/src# python3 upload -H 10.0.0.5 -p 5005 -s /tmp/archivo-500KB.bin -n archivo-500KB.bin -r sack
File '/tmp/archivo-500KB.bin' sent successfully using Selective Repeat protocol.
root@compu-atu:/home/atuefullana/Desktop/redes/TP1/src#
```

Figura 6: Notificación de transferencia exitosa en el cliente y métricas registradas por el servidor.

4.2.2. Variante SACK

Al ejecutar la transferencia utilizando el protocolo SACK el procedimiento es completamente análogo, con la diferencia de que el cliente recibe la confirmación explícita de que el archivo se subió con éxito mediante el mecanismo de SACK.

4.3. Concurrencia

```
"Node: h2" "Node: h3"
root@compu-atu:/home/atuefullana/Desktop/redes/TP1/src# python3 upload -H 10.0.0.5 -p 5005 -s /tmp/archivo-2MB.bin -n archivo-2MB.bin -r sw
File '/tmp/archivo-2MB.bin' sent successfully using Stop-and-Wait protocol.
root@compu-atu:/home/atuefullana/Desktop/redes/TP1/src#
root@compu-atu:/home/atuefullana/Desktop/redes/TP1/src# python3 upload -H 10.0.0.5 -p 5005 -s /tmp/archivo-500KB.bin -n archivo-500KB.bin -r sack
File '/tmp/archivo-500KB.bin' sent successfully using Selective Repeat protocol.
root@compu-atu:/home/atuefullana/Desktop/redes/TP1/src#
```

Figura 7: Lanzamiento de transferencias simultáneas desde los hosts cliente 2 y 3.

Para validar el manejo de múltiples conexiones simultáneas, desde el cliente 2 se envía la solicitud para subir un archivo de 2 MB en el servidor utilizando *Stop & Wait*. Inmediatamente después, desde el cliente 3 se manda una solicitud para subir un archivo mas chico (500 KB) utilizando SACK.

```
"Node: h5"
root@compu-atu:/home/atuefullana/Desktop/redes/TP1/src# python3 start-server -H 10.0.0.5 -p 5005
FIN recibido: seq=489
Transferencia completada: 500000 bytes
FIN recibido: seq=1954
Transferencia completada: 2000000 bytes
█
```

Figura 8: Finalización de la subida del cliente 3 con antes a la del cliente 2.

Como se ve, por más que el cliente 2 fue el primero en iniciar la transmisión, la subida del archivo correspondiente al cliente 3 termino antes. Esto demuestra que el servidor procesa las solicitudes de manera concurrente y aplica para cada flujo el protocolo acordado individualmente con cada cliente.

No.	Time	Source	Destination	Protocol	Length	Info
1084	15.482711038	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=837 Ack=836 Len=1024
1085	15.484331084	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=837 Ack=838 Len=0
1086	15.487708137	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=838 Ack=837 Len=1024
1087	15.489294642	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=838 Ack=839 Len=0
1088	15.492678322	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=839 Ack=838 Len=1024
1089	15.494318195	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=839 Ack=840 Len=0
1090	15.497725169	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=840 Ack=839 Len=1024
1091	15.499307110	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=840 Ack=841 Len=0
1092	15.502692000	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=841 Ack=840 Len=1024
1093	15.504287513	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=841 Ack=842 Len=0
1094	15.507670013	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=842 Ack=841 Len=1024
1095	15.509257519	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=842 Ack=843 Len=0
1096	15.512629938	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=843 Ack=842 Len=1024
1097	15.514281972	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=843 Ack=844 Len=0
1098	15.517596521	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=844 Ack=843 Len=1024
1099	15.519824749	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=844 Ack=845 Len=0
1700	15.522238414	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=845 Ack=844 Len=1024
1701	15.523432999	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=845 Ack=846 Len=0
1702	15.526547566	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=846 Ack=845 Len=1024
1703	15.527744076	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=846 Ack=847 Len=0
1704	15.531099266	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=847 Ack=846 Len=1024
1705	15.532172017	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=847 Ack=848 Len=0
1706	15.535299210	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=848 Ack=847 Len=1024
1707	15.536450937	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=848 Ack=849 Len=0
1708	15.539674759	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=849 Ack=848 Len=1024
1709	15.540842685	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=849 Ack=850 Len=0
1710	15.543878083	00:00:00_00:00:03	Broadcast	ARP	42	who has 10.0.0.5? Tell 10.0.0.3
1711	15.544131933	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=850 Ack=849 Len=1024
1712	15.544947614	00:00:00_00:00:05	00:00:00_00:00:03	ARP	42	10.0.0.5 is at 00:00:00:00:00:05
1713	15.545421628	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=850 Ack=851 Len=0
1714	15.548378544	10.0.0.3	10.0.0.5	SACK	70	START Seq=0 Ack=0 Len=17
1715	15.548516113	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=851 Ack=850 Len=1024
1716	15.548544992	10.0.0.5	10.0.0.3	SACK	53	ACK Seq=0 Ack=0 Len=0
1717	15.549972163	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=851 Ack=852 Len=0
1718	15.553129344	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=0 Ack=0 Len=1024
1719	15.553132576	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=852 Ack=851 Len=1024
1720	15.553133194	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=1 Ack=0 Len=1024
1721	15.553133380	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=2 Ack=0 Len=1024
1722	15.553134646	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=3 Ack=0 Len=1024
1723	15.553145756	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=4 Ack=0 Len=1024
1724	15.553146086	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=5 Ack=0 Len=1024
1725	15.553146369	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=6 Ack=0 Len=1024
1726	15.553148614	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=7 Ack=0 Len=1024
1727	15.553150964	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=8 Ack=0 Len=1024
1728	15.553153661	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=9 Ack=0 Len=1024
1729	15.553156465	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=10 Ack=0 Len=1024
1730	15.553159295	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=11 Ack=0 Len=1024
1731	15.553165947	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=12 Ack=0 Len=1024

Figura 9: Captura en Wireshark del arribo del paquete **START** del cliente 3 durante la sesión activa del cliente 2.

Al analizar en Wireshark el momento exacto en el que arriba el paquete de inicio (**START**) del cliente 3, se puede constatar en detalle cómo el servidor atiende concurrentemente a los dos clientes, y cómo el archivo del cliente 3 completa su transferencia más rápido debido a su menor tamaño y al mayor rendimiento del **SACK**.

No.	Time	Source	Destination	Protocol	Length	Info
1769	15.555268812	10.0.0.5	10.0.0.3	SACK	53	ACK Seq=17 Ack=18 Len=0
1770	15.555276264	10.0.0.5	10.0.0.3	SACK	53	ACK Seq=18 Ack=19 Len=0
1771	15.555285283	10.0.0.5	10.0.0.3	SACK	53	ACK Seq=19 Ack=20 Len=0
1772	15.555292698	10.0.0.5	10.0.0.3	SACK	53	ACK Seq=20 Ack=21 Len=0
1773	15.555299940	10.0.0.5	10.0.0.3	SACK	53	ACK Seq=21 Ack=22 Len=0
1774	15.555308869	10.0.0.5	10.0.0.3	SACK	53	ACK Seq=22 Ack=23 Len=0
1775	15.555316403	10.0.0.5	10.0.0.3	SACK	53	ACK Seq=23 Ack=24 Len=0
1776	15.555323637	10.0.0.5	10.0.0.3	SACK	53	ACK Seq=24 Ack=25 Len=0
1777	15.555333068	10.0.0.5	10.0.0.3	SACK	53	ACK Seq=25 Ack=26 Len=0
1778	15.555340451	10.0.0.5	10.0.0.3	SACK	53	ACK Seq=26 Ack=27 Len=0
1779	15.555347523	10.0.0.5	10.0.0.3	SACK	53	ACK Seq=27 Ack=28 Len=0
1780	15.555354577	10.0.0.5	10.0.0.3	SACK	53	ACK Seq=28 Ack=29 Len=0
1781	15.555363662	10.0.0.5	10.0.0.3	SACK	53	ACK Seq=29 Ack=30 Len=0
1782	15.555371121	10.0.0.5	10.0.0.3	SACK	53	ACK Seq=30 Ack=31 Len=0
1783	15.555378412	10.0.0.5	10.0.0.3	SACK	53	ACK Seq=31 Ack=32 Len=0
1784	15.558174864	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=853 Ack=852 Len=1024
1785	15.559296133	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=853 Ack=854 Len=0
1786	15.562439899	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=854 Ack=853 Len=1024
1787	15.563578524	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=854 Ack=855 Len=0
1788	15.566697557	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=855 Ack=854 Len=1024
1789	15.567869507	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=855 Ack=856 Len=0
1790	15.571001519	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=856 Ack=855 Len=1024
1791	15.572153278	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=856 Ack=857 Len=0
1792	15.575277757	10.0.0.2	10.0.0.5	STPOWAIT	1077	DATA Seq=857 Ack=856 Len=1024
1793	15.576487459	10.0.0.5	10.0.0.2	STPOWAIT	53	ACK Seq=857 Ack=858 Len=0
1794	15.578713497	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=32 Ack=0 Len=1024
1795	15.578723627	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=33 Ack=0 Len=1024
1796	15.578726270	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=34 Ack=0 Len=1024
1797	15.578736313	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=35 Ack=0 Len=1024
1798	15.578737096	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=36 Ack=0 Len=1024
1799	15.578737895	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=37 Ack=0 Len=1024
1800	15.578738551	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=38 Ack=0 Len=1024
1801	15.578739155	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=39 Ack=0 Len=1024
1802	15.578740618	10.0.0.3	10.0.0.5	SACK	1077	DATA Seq=40 Ack=0 Len=1024

Figura 10: Evidencia de tráfico concurrente de ambos clientes atendidos simultáneamente por el servidor.

En la figura 10 se evidencia con nitidez el intercambio simultáneo y multiplexado de datagramas correspondiente a ambas sesiones de transferencia.

4.4. Inspección de Tráfico y Eficiencia en Wireshark

No.	Time	Source	Destination	Protocol	Length	Info
75	12.177748552	10.0.0.5	10.0.0.2	STOPWAIT	53	ACK Seq=26 Ack=27 Len=0
76	12.188290255	10.0.0.2	10.0.0.5	STOPWAIT	1077	DATA Seq=27 Ack=26 Len=1024
77	13.189697231	10.0.0.2	10.0.0.5	STOPWAIT	1077	DATA Seq=27 Ack=26 Len=1024
78	14.191061980	10.0.0.2	10.0.0.5	STOPWAIT	1077	DATA Seq=27 Ack=26 Len=1024
79	14.222582791	10.0.0.5	10.0.0.2	STOPWAIT	53	ACK Seq=27 Ack=28 Len=0

Figura 11: Recuperación ante pérdida de paquetes y temporización de retransmisión en *Stop & Wait*.

En la figura 11 se observa cómo el protocolo *Stop & Wait* resuelve la pérdida de paquetes y gestiona la posterior retransmisión. En la columna *Time* podemos mirar que, antes de reenviar el paquete descartado, el protocolo aguarda 1 segundo, correspondiente al valor establecido para el *timeout*. Una vez vence el tiempo, se procede a retransmitir el paquete.

En el entorno simulado con Mininet se conocen con certeza el (*delay*) y el (*packet loss*); sin embargo, en una red real estos parámetros son variables. Entonces, el protocolo poder tener en cuenta la posibilidad de que el ACK simplemente esté en tránsito con delay, evitando retransmitir antes de tiempo para no generar congestión innecesaria en el canal.

4.5. Validación de Requisitos Críticos de Rendimiento

- **Tolerancia a tasa de pérdida del 10 %:** Transferencia validada sin corrupción bajo enlaces con packet loss.
- **Operación con RTT variable (hasta 300 ms):** Llegada y envío de segmentos bajo alta latencia.

4.5.1. Cumplimiento del requisito crítico: Transferencia de 5 MB en menos de 2 minutos

Al finalizar el envío del archivo de 5 MB utilizando el protocolo SACK, con un 10% de *packet loss* y un RTT de 40 ms, se puede observar en la consola del servidor (Figura 12) que se notifica que el último número de secuencia fue el 4883. Esto tiene sentido sabiendo que nuestro protocolo sólo puede enviar 1024 bytes de *payload* por paquete y el archivo pesaba 5.000.000 bytes; si realizamos el cálculo obtenemos $5,000,000/1024 \approx 4883$ paquetes.

```
"Node: h5"
root@compu-atu:/home/atuefullana/Desktop/redes/TP1/src# python3 start-server -
H 10.0.0.5 -p 5005
FIN recibido: seq=4883
Transferencia completada: 5000000 bytes
```

Figura 12: Consola del servidor indicando el final de la transferencia del archivo de 5 MB y el último número de secuencia (4883).

Como podemos ver en la captura en Wireshark (Figura 13), donde filtramos por el paquete con *OPCODE = START* que inicia el *handshake* con el servidor, y también filtramos el último paquete con el número de secuencia 4883, al observar la columna de *Time* confirmamos que el protocolo tardó menos de 2 minutos en finalizar la transferencia cumpliendo el límite.

No.	Time	Source	Destination	Protocol	Length	Info
3	3.075089485	10.0.0.2	10.0.0.5	STOPWAIT/SACK	68	START Seq=0 Ack=0 Len=15
9592	89.398796433	10.0.0.2	10.0.0.5	STOPWAIT/SACK	53	FIN Seq=4883 Ack=0 Len=0
9593	89.429578917	10.0.0.5	10.0.0.2	STOPWAIT/SACK	53	ACK Seq=4883 Ack=4883 Len=0

Figura 13: Captura de tráfico en Wireshark evidenciando el inicio y fin de la transferencia en un tiempo menor a 2 minutos.

5. Preguntas a Responder

5.1. Pregunta 1: Arquitectura Cliente-Servidor

Enunciado: Describa la arquitectura Cliente-Servidor.

En esta arquitectura, los clientes son los encargados de iniciar la comunicación con el servidor enviando solicitudes, mientras que los servidores tienen el rol pasivo de escuchar y responder a dichas peticiones para establecer la comunicación y proveer el servicio o recurso al cual el cliente desea acceder. En nuestra implementación, dichos servicios corresponden a las operaciones de **DOWNLOAD** y **UPLOAD**.

El servidor debe ser capaz de atender a múltiples clientes de forma concurrente, garantizando que los clientes no se comuniquen entre sí, sino exclusivamente con el servidor central. La conexión permite una comunicación continua y bidireccional; es decir, tanto el servidor como el cliente pueden emitir y recibir datos mutuamente a lo largo de la sesión.

En el sistema implementado, el servidor permanece a la escucha en un puerto UDP determinado. Dado que UDP es un protocolo no orientado a conexión y sin estado a nivel de transporte, el servidor identifica a cada cliente mediante la tupla formada por su dirección IP y puerto de origen, manteniendo una estructura de estado por cliente que define su sesión activa durante la transferencia.

5.2. Pregunta 2: Función de un Protocolo de Capa de Aplicación

Enunciado: ¿Cuál es la función de un protocolo de capa de aplicación?

La función de un protocolo de capa de aplicación es definir cómo se comunican dos procesos que se ejecutan en diferentes dispositivos terminales a través de la red. Se encarga de especificar:

- Los tipos de mensajes que se intercambian (por ejemplo, peticiones, confirmaciones y mensajes de error).
- La sintaxis de cada mensaje (los campos presentes, su disposición y su tamaño en bytes).
- La semántica de los campos (el significado concreto de la información transmitida).
- Las reglas que determinan cuándo y cómo un proceso emite o responde a los mensajes recibidos.

5.3. Pregunta 3: Detalle del Protocolo de Aplicación Desarrollado

Enunciado: Detalle el protocolo de aplicación desarrollado en este trabajo.

El protocolo de aplicación diseñado para este trabajo estructura su comunicación binaria definiendo diversos tipos de mensajes (OPCODES) empotrados en el primer byte (**Type**) del header del datagrama. Los mensajes soportados son:

- **START/HANDSHAKE (0):** Solicitud inicial para negociar la operación (**UPLOAD** / **DOWNLOAD**), la variante a utilizar (**SACK** o **Stop & Wait**), y los metadatos como el nombre y tamaño del archivo.
- **DATA (1):** Mensaje portador de fragmentos o bloques binarios del archivo a través del campo **Payload**.
- **ACK (2):** Mensaje emitido por el receptor para acusar la recepción acumulativa e indicar qué secuencia de paquete espera a continuación.
- **SACK (3):** Mensaje de confirmación análogo al **ACK** pero que adjunta, dentro del campo de longitud variable **SackData**, bloques y rangos que especifican paquetes recibidos fuera de orden.
- **ERROR (4):** Utilizado para notificar condiciones anormales al otro extremo, como la solicitud de un archivo inexistente.

- **FIN (5):** Mensaje encargado de señalar el final de la transferencia, habilitando un cierre limpio de la sesión por ambas partes.

Para la correcta generación y compresión de estos mensajes a nivel binario se utilizó la librería `struct` de Python. Ésta garantiza que los campos numéricos del header —tales como `SeqNum` y `AckNum` de 4 bytes, o `Length` de 2 bytes— mantengan el formato estricto estipulado independientemente de la arquitectura nativa del ordenador, empaquetándose en formato *big-endian* (*network byte order*) (ej., `'!B I I H'`).

5.4. Pregunta 4: Características y Comparativa de TCP y UDP

Enunciado: La capa de transporte del stack TCP/IP ofrece dos protocolos: TCP y UDP. ¿Qué servicios proveen dichos protocolos? ¿Cuáles son sus características? ¿Cuándo es apropiado utilizar cada uno?

Ambos protocolos proveen como mecanismo básico la detección de errores de extremo a extremo mediante el campo de suma de verificación (*checksum*).

- **TCP (Transmission Control Protocol):** Es un protocolo orientado a la conexión que requiere una fase previa de negociación (*three-way handshake*). Provee una transferencia confiable de datos que garantiza la entrega, preserva el orden secuencial de los bytes y descarta duplicados mediante números de secuencia, confirmaciones (ACKs) y retransmisiones automáticas. Además, incorpora mecanismos de control de flujo (ventana del receptor) y control de congestión (ajuste de la ventana según el estado del canal). Es el protocolo adecuado cuando no se tolera la pérdida de información y se prioriza la integridad por sobre la latencia. Ejemplos de uso: navegación web (HTTP/HTTPS), transferencia de archivos (FTP, SFTP), correo electrónico (SMTP/IMAP) y sistemas de transacciones bancarias.
- **UDP (User Datagram Protocol):** Es un protocolo simple, liviano y no orientado a la conexión (*connectionless*), sin negociación previa. No garantiza la entrega de los datagramas, el orden de llegada ni la eliminación de duplicados, y carece de retransmisión nativa, control de flujo o control de congestión. Resulta óptimo cuando una aplicación tolera cierta pérdida eventual de paquetes a cambio de una mínima latencia y menor sobrecarga de headers. Ejemplos de uso: videojuegos multijugador en tiempo real, telefonía sobre IP (VoIP), videollamadas, streaming multimedia y servicios de consulta rápida como DNS.

5.5. Pregunta 5: Mecanismos de Control de Congestión

Enunciado: Justifique si el protocolo desarrollado cuenta con mecanismos de control de congestión; en caso de tenerlos, descríbalos.

El protocolo desarrollado **no cuenta con control de congestión**. La ventana del modo SACK tiene un tamaño fijo que no crece ni disminuye según el estado de la red, y el *timeout* de retransmisión es igualmente un valor fijo que no utiliza ningún algoritmo para calcular dinámicamente el RTT.

Si se produce la pérdida de un paquete, el protocolo únicamente se limita a retransmitirlo, sin evaluar si la red se encuentra congestionada o si los buffers intermedios están saturados. El protocolo retransmite hasta que llega al fin del archivo o hasta que el *timeout* vence 5 veces consecutivas (o el MAXTRIES definido), momento en el que se aborta la comunicación.

5.6. Pregunta 6: Comparación de Tiempos entre Protocolos

Enunciado: ¿Cuál de los dos protocolos desarrollados envía archivos en la menor cantidad de tiempo? ¿Es siempre el mismo?

Protocolo	Tamaño	RTT (ms)	Pérdida (%)	Tiempo (s)	Throughput (KB/s)
Stop & Wait	500 KB	40 ms	0 %	20,36	24,56
Stop & Wait	500 KB	40 ms	10 %	127,47	3,92
SACK	500 KB	40 ms	0 %	1,09	458,72
SACK	500 KB	40 ms	10 %	8,18	61,12
Stop & Wait	2 MB	40 ms	0 %	81,29	24,60
Stop & Wait	2 MB	40 ms	10 %	515,02	3,88
SACK	2 MB	40 ms	0 %	4,03	496,28
SACK	2 MB	40 ms	10 %	34,15	58,56
Stop & Wait	5 MB	40 ms	10 %	1.342,34	3,72
SACK	5 MB	40 ms	10 %	85,45	58,51

Tabla 2: Comparación de tiempos de transferencia y cálculo de Throughput promedio.

Como se puede ver en la tabla el protocolo que envía archivos en la menor cantidad de tiempo es el **SACK**. Esto tiene sentido ya que *Stop & Wait* sólo puede enviar un paquete a la vez y se queda esperando la confirmación de que ese único paquete llegó, mientras no envía más. Mientras que **SACK** tiene una ventana de 32 paquetes, lo que le permite enviar muchos paquetes sin esperar a un ACK. El único caso en el que **SACK** y **Stop & Wait** envían al mismo tiempo es cuando se envía un único paquete.

6. Dificultades Encontradas

Se presentaron los siguientes desafíos:

- **Entender el problema planteado en el enunciado y el uso de las herramientas:** Pasar de únicamente tener clases teoricas y practicas a encarar el proyecto de forma grupal fue una cuesta arriba. Conocer de los conceptos de la clase pero no saber del todo como aplicarlos. Plantear cosas como la necesidades minimas del trabajo antes de poder dividirnos las tareas. No solo eso, Mininet era algo nuevo para nosotros, y el uso de extensiones en wireshark también.
- **Atender a los clientes de forma concurrente:** Administrar transferencias en paralelo sobre sockets UDP no orientados a conexión requirió tomar como distintos el estado de cada sesión.
- **Creación del plugin de Wireshark utilizando un lenguaje de programación nuevo:** La extension para Wireshark demandó aprender y familiarizarse con una nueva tecnología(Lua) para parsear adecuadamente los headers del protocolo propio.
- **Generar abstracción entre protocolos:** Diseñar una arquitectura limpia y modular que permita desacoplar la lógica de *Stop & Wait* y de *TCP con SACK*, facilitando la reutilización del código de transporte y manejo de sockets sin duplicación de lógica.
- **Debugeo de errores:** Identificar y corregir fallos temporales, pérdidas de paquetes inducidas y condiciones de carrera en un entorno distribuido emulado como Mininet.
- **Implementación de Selective Repeat:** Manejo de la complejidad de la ventana deslizante con SACK, incluyendo la gestión de buffers de recepción fuera de orden, el rastreo de huecos (*gaps*) y la retransmisión puntual de los paquetes descartados.

7. Conclusión

A partir del desarrollo y evaluación de este sistema de transferencia, se pudo evidenciar de manera concluyente la notable diferencia de eficiencia entre los dos esquemas implementados. La segmentación introducida por la variante TCP con SACK demostró ser superior frente al enfoque de *Stop & Wait*, especialmente en escenarios caracterizados por un producto retardo-ancho de banda elevado. Al mantener múltiples paquetes en paralelo, el mecanismo SACK permite hacer eficiente la utilización del canal, logrando tiempos de transferencia más bajos en enlaces con latencia y mostrando como impacta el tamaño de ventana en la red.

No obstante, esta indiscutible mejora en el rendimiento conllevó a un costo directo sobre la complejidad de la arquitectura lógica. Mientras que el esquema *Stop & Wait* resultó predecible y directo de implementar gracias a su naturaleza síncrona y paso a paso, la integración del *Selective Repeat* exigió un diseño mucho más sofisticado. Fue necesario programar estructuras complejas para gestionar la ventana deslizante, asignar buffers dinámicos para almacenar paquetes fuera de orden, rastrear con precisión los "huecos" (*gaps*) notificados y orquestar retransmisiones altamente puntuales. Este contraste deja en evidencia el permanente compromiso (*trade-off*) que existe en la ingeniería de protocolos: incrementar la eficiencia implica, inevitablemente, acrecentar el nivel de complejidad algorítmica y los recursos de procesamiento requeridos.

Finalmente, el comportamiento empírico del sistema respaldó exitosamente las hipótesis de diseño trazadas en la fase inicial, superando satisfactoriamente las métricas críticas del protocolo. En las pruebas más exigentes, soportando un entorno adverso con tasas de pérdida constante del 10 % y demoras que elevaron el RTT por encima de los 300 ms, el protocolo garantizó la reconstrucción de los archivos respetando su integridad original sin corrupción de datos ni interrupciones abruptas no controladas. Quedó comprobado con firmeza que, mediante una correcta implementación a nivel de capa de aplicación, es plenamente posible proveer un servicio de transferencia de datos altamente robusto y confiable tomando como cimiento un protocolo *best-effort* como UDP.